

IPO Manager

A Complete Learning Guide for Non-Coders

Understanding every folder, library, and decision behind a real Electron desktop application

From the encrypted vault to the Playwright automation,
from the master password to the AU IPO bid submission —
explained in plain English for a CA learning to code.

Version 0.1.0 · May 2026

Built and documented with the help of Claude (Anthropic).

Table of Contents

Table of Contents	3
Preface — How to Read This Guide	8
Who this guide is for	8
How to read it	8
A note on screenshots	8
Part 1 — The Big Picture	9
1.1 What is IPO Manager?	9
1.2 The problem it solves	9
1.3 Who uses it?	9
1.4 What does it look like conceptually?	10
Part 2 — Foundational Concepts for Non-Coders	11
2.1 What is a desktop application?	11
2.2 The three worlds: Frontend, Backend, Database	11
2.3 What is Electron?	11
2.4 JavaScript, TypeScript, Node.js — what's the difference?	12
2.5 What is encryption?	12
2.6 What are libraries / packages?	13
Part 3 — How the App Works End-to-End	14
3.1 The journey of your master password (unlock flow)	14
3.2 The journey of a bank login (e.g. AU Bank for Akshay)	15
3.3 The journey of an AU IPO bid (multi-member)	16
Part 4 — The Project Folder Structure	17
4.1 package.json — the project manifest	17
4.2 src/ — the source code	18
4.3 node_modules/ — the downloaded libraries	22
4.4 dist/ — the build output	22
4.5 out/ — the compiled JavaScript	23
4.6 tests/ — automated tests	23

4.7 Configuration files	23
-----------------------------------	----

Part 5 — Every Library, Explained
25

5.1 electron — the desktop app framework	25
5.2 react + react-dom — the UI framework	25
5.3 typescript — the type-safety layer	25
5.4 playwright — the browser automation engine	25
5.5 better-sqlite3-multiple-ciphers — encrypted SQLite	26
5.6 argon2 — the password hashing library	26
5.7 keytar — the OS keychain bridge	26
5.8 googleapis — the Gmail API client	27
5.9 electron-vite + vite + electron-builder — the build chain	27
5.10 Specialty libraries	27
5.11 vitest — the test runner	27

Part 6 — The Security Model
28

6.1 Defense in depth	28
6.2 The three encryption layers	28
6.3 Where each key lives	28
6.4 Threat model — who are we protecting against?	28
6.5 What we DO protect against	29
6.6 What we DON'T protect against	30
6.7 What leaves the machine — explicitly listed	30

Part 7 — Features: What Works
31

7.1 Vault setup and unlock	31
7.2 Family and member management	31
7.3 Bank login automation	31
7.4 Broker login + portfolio fetch	31
7.5 AU Bank IPO bidding	32
7.6 Excel import / export	32
7.7 Encrypted incremental backup	32
7.8 Member detail card (the credentials cockpit)	32
7.9 Spreadsheet view	33
7.10 Cost-safety guardrails for CAPTCHA solving	33
7.11 Manual lock + Ctrl+L	33

7.12 Factory reset	33
Part 8 — Limitations and Known Issues	34
8.1 Selector brittleness — websites change	34
8.2 Only AU IPO bidding is automated	34
8.3 No allotment tracking	34
8.4 No auto-bid scheduler	34
8.5 No mobile app	34
8.6 Browser-dependent	34
8.7 No code signing	35
8.8 No auto-update	35
8.9 Logs grow unbounded	35
8.10 No DB indexes on some columns	35
Part 9 — Security Risks — Current Posture	36
9.1 Critical risks — now mitigated	36
9.2 Remaining risks (yellow, not red)	36
9.3 What could go wrong — failure scenarios	36
Part 10 — One Click, End to End	38
Step 1: The click is dispatched in React	38
Step 2: loginBank() prepares the IPC call	38
Step 3: The preload bridge	38
Step 4: Main process receives the message	39
Step 5: runLogin loads credentials from the database	39
Step 6: launchSession opens a real Chromium window	39
Step 7: The adapter runs the actual login	39
Step 8: Balance is fetched	40
Step 9: Audit log is written	40
Step 10: Return to renderer	40
Step 11: UI updates	40
Part 11 — Glossary	41
Part 12 — How to Add a New Bank Adapter	43
12.1 Pick a code	43

12.2 Create the adapter file	43
12.3 Inspect the real bank's login flow	43
12.4 Implement login() step by step	44
12.5 Implement fetchBalance()	44
12.6 Register the adapter	44
12.7 Add the OTP query	44
12.8 Add to the BANKS constant in the renderer	45
12.9 Build and test	45
12.10 Common gotchas	45

Part 13 — Common Patterns in the Codebase 46

13.1 The IPC handler pattern	46
13.2 The encrypted-field pattern	46
13.3 The adapter interface pattern	46
13.4 The bus / channel naming convention	47
13.5 The 'busy state' pattern in the renderer	47
13.6 The toast notification pattern	47
13.7 The 'never throw, always return' contract	47
13.8 The 'sensitive data lives in BLOBs, display data lives in TEXT' pattern	48

Part 14 — Data Flow Diagrams 49

14.1 The unlock flow	49
14.2 The encrypted backup flow	50
14.3 The CAPTCHA solve flow with cost gate	51

Appendix A — Command Cheat Sheet 52

Building	52
Backup folder management	52
Inspecting the vault file	52
Diagnosing automation failures	52

Appendix B — Troubleshooting 53

Symptom: "Wrong password" after typing correctly	53
Symptom: Clicking a bank does nothing	53
Symptom: CAPTCHA solving keeps failing	53
Symptom: Backup pill is yellow or red	53

Symptom: "Could not decrypt the field key with the master password" when restoring 53

Symptom: App is very slow to start 53

Where to ask for help 54

Closing Note **55**

Preface — How to Read This Guide

Bhai, namaste. If you are reading this, you have either commissioned this app for your own use (in which case you already know the financial pain it solves), or you are looking at the codebase wondering where to start. Either way, the goal of this document is the same: **take you from zero programming knowledge to a confident understanding of how every piece of this application fits together.**

You are a Chartered Accountant. You think in terms of double-entry, reconciliation, regulatory compliance, and risk. That mental model is closer to programming than you might think — both are about *moving information from one place to another while preserving integrity*. Throughout this guide, when I introduce a programming concept, I will use an accounting or finance analogy wherever possible.

Who this guide is for

- **Non-coders** who own this app and want to understand what is inside.
- **Learners** who want to see how a real desktop application is structured.
- **Future contributors** (you, or anyone you hire) who want to add features.
- **Auditors / security-conscious users** who want to know what data goes where.

How to read it

The guide is organized in eleven Parts, ordered from most-conceptual to most-detailed. If you read end-to-end (recommended), it will take 3–4 hours. Parts 1–3 give you the big picture and answer *what does this do, and how*. Parts 4–5 explain *where everything lives on disk and why*. Parts 6–9 cover the security model, features, limitations, and risks. Part 10 traces a single click through the entire codebase as a worked example. Part 11 is a glossary you can refer back to.

A note on screenshots

This document uses ASCII layouts (text-based diagrams) in monospace font to depict UI and folder structures. If you want, you can take real screenshots of your own machine and paste them next to the descriptions in your own annotated copy. The ASCII layouts are precise — they show exactly what you would see in the app or in your file explorer.

Tip: Read this guide with the actual app open in one window and your file explorer open in another, navigating to each folder as it is described. Programming is a hands-on subject. The fastest way to understand a folder is to click into it.

Part 1 — The Big Picture

1.1 What is IPO Manager?

IPO Manager is a Windows desktop application that helps a Chartered Accountant (or any individual managing IPO applications for a large family) keep track of dozens of bank accounts, demat accounts, and IPO applications across multiple relatives — all in one encrypted vault.

It does three big things:

- **Stores credentials** for every relative's banks (AU, YES, SBI, KOTAK, ICICI, BOB, PNB, HDFC, AXIS) and brokers (Zerodha, Dhan, Angel One, Mirae, Shoonya, Fyers, Groww) — encrypted at rest, never sent anywhere.
- **Automates logins** — clicking a bank's name in the app opens a real browser, fills in user-id + password, fetches the OTP from Gmail, solves the CAPTCHA via Claude AI, and shows you the current balance.
- **Submits IPO bids on AU Bank** for any selected subset of family members — instead of logging in 35 times during a hot IPO window.

1.2 The problem it solves

A practising CA's family commonly has 30+ demat accounts across multiple relatives (spouse, parents, in-laws, HUF, children once they turn 18). When a hot IPO opens, you have a 3-day window to apply on behalf of each one — and you want to maximize allotment chances, so each family member should apply separately.

Doing this manually means:

- Logging into 8 different bank websites, 30 times each — that's 240 logins.
- Each login needs a password (different per bank), maybe an OTP (different per bank, sent to different mobile numbers / emails), and sometimes a CAPTCHA.
- Once logged in, you navigate three or four menus to find the IPO bid page, enter the right amount, the right UPI or ASBA, and submit.
- If the bid fails (wrong amount, technical glitch, balance below minimum), you redo it.

This is a full week of work for one IPO. IPO Manager compresses it to a half-hour, with the user staying in control (the bid is shown for confirmation before submission — the app never silently sends money anywhere).

1.3 Who uses it?

Currently: **one user** — the owner who commissioned this app. The architecture is single-user by design: there is one master password, one vault, one user's data on the machine. Multiple-machine sync (Phase 4) lets the same user run the app at home and at the office, with the vault synchronised through OneDrive or Google Drive.

A second-user scenario (e.g. handing the installer to a CA friend) is supported, but each user has their *own* independent vault. There is no shared cloud service, no multi-tenant SaaS — this is intentional. The whole point of a local-only encrypted vault is to avoid the regulatory and security overhead of cloud-hosted credentials.

1.4 What does it look like conceptually?

The app is a single window. The left side is a sidebar with families and tools. The right side is the main panel, which shows either the all-members accordion view, a single-family deep view, or the spreadsheet grid view.

[illegible]

Everything you see here is rendered by the React framework running inside an Electron window. We will get to what that means in Part 2.

Part 2 — Foundational Concepts for Non-Coders

This Part introduces the building blocks. If you already know these terms, skim it. If you do not, read carefully — every later Part assumes you understand this vocabulary.

2.1 What is a desktop application?

A **desktop application** is a program you double-click on Windows or macOS to run. It opens in its own window. Examples: Microsoft Word, Tally, Chrome, VLC media player. Contrast this with:

- **Web applications** — you visit a URL in a browser; the program runs on a remote server (Gmail, ICICI net banking, Quickbooks Online).
- **Mobile applications** — you install from Play Store or App Store; the program runs on your phone.

IPO Manager is a desktop application. It runs entirely on your Windows PC. There is no remote server holding your data. Your vault lives in a single file on your hard disk.

Why desktop and not web? Banks and brokers do not have APIs we can call directly — we have to drive their websites with a real browser. A desktop app can launch and control a real browser; a web app cannot. Also, keeping credentials off any server eliminates an entire category of breach risk.

2.2 The three worlds: Frontend, Backend, Database

Almost every application — desktop, web, or mobile — is structured as three cooperating layers:

Layer	What it does	Where it lives	In IPO Manager
Frontend	Draws the UI you see; handles clicks and keyboard input	Renderer process	src/renderer/ — React + TypeScript
Backend	Runs the actual work — calls APIs, drives browser, encrypts/decrypts data, talks to the OS	Main process	src/main/ — TypeScript with Playwright + crypto
Database	Stores data persistently between runs	Hard disk (an encrypted file)	vault.db — SQLite + SQLCipher

The Frontend never touches your credentials directly. The Renderer process is sandboxed — it cannot open files on your disk or make network requests by itself. When you click "Refresh AU", the Renderer sends a message to the Main process: "please run the AU login for Akshay". The Main process does the real work, then sends the result back. This separation is a security feature.

2.3 What is Electron?

Electron is a framework — a collection of pre-built components — that lets you build a desktop application using web technologies (HTML, CSS, JavaScript). It bundles together:

- **Chromium** — the open-source browser engine (the same engine inside Chrome) — to render the user interface.
- **Node.js** — a JavaScript runtime — to run the backend logic.
- **An IPC bridge** — Inter-Process Communication — so the frontend (Chromium) and backend (Node) can send messages to each other.

Why is this a big deal? Without Electron, building a Windows desktop app would mean learning C# or C++ — heavy languages with their own toolchains. Electron lets you use JavaScript everywhere. Slack, VS Code, Discord, WhatsApp Desktop, Signal — all are Electron apps.

Trade-off: Electron apps are bigger (they bundle Chromium = ~80 MB) and use more memory than native apps. For IPO Manager this is irrelevant — you have a desktop PC with plenty of RAM.

2.4 JavaScript, TypeScript, Node.js — what's the difference?

JavaScript is the programming language that web pages use. It was invented in 1995 for use inside a browser. Today it runs everywhere — browsers, servers, phones, even Mars rovers.

Node.js is JavaScript running outside the browser. Specifically, it can read and write files, listen on network ports, spawn child processes — things a browser JavaScript engine cannot do. When you see code that imports 'node:fs' or 'node:path', that is Node-specific.

TypeScript is JavaScript plus type annotations. In plain JavaScript:

```
function add(a, b) { return a + b; }
add(2, 'hello'); // returns '2hello' – surprising!
```

In TypeScript:

```
function add(a: number, b: number): number {
  return a + b;
}
add(2, 'hello'); // compiler error before the program even runs
```

TypeScript catches a huge class of bugs *before* the program runs. The trade-off is that you have to write the type annotations. For a financial app like ours, where a misplaced character could leak a password or transfer the wrong amount, this safety net is worth the effort.

Every .ts file in IPO Manager gets compiled (transformed) into plain JavaScript before it actually runs. You can see the compiled output in the out/ folder.

2.5 What is encryption?

Encryption is the process of scrambling information so that only someone with a secret key can unscramble it. There are two main flavours:

- **Symmetric encryption** — one key. Same key encrypts and decrypts. Used here for the vault (AES-256-GCM, SQLCipher).
- **Asymmetric (public-key) encryption** — two keys, a public one and a private one. Used here only indirectly (HTTPS connections to banks).

Three pieces of jargon you will see repeatedly:

- **Plaintext** — readable, unencrypted data. "akshay@gmail.com" is plaintext.
- **Ciphertext** — encrypted data. Looks like random bytes: a3 7f 92 ee 01 ...
- **Key** — the secret used to encrypt and decrypt. In our app, the key is derived from your master password using a function called Argon2id (more on that in Part 6).

A common beginner question: *if my master password derives the key, why not just store the password and use it directly?* Two reasons:

- Argon2id is intentionally slow (~500 ms per derivation). This makes it expensive for an attacker to try millions of passwords against a stolen vault file.
- The derived key is exactly 32 bytes — the size SQLCipher requires. Passwords are any length. So we still need a key-derivation step.

2.6 What are libraries / packages?

A **library** (also called a **package**) is pre-written code that solves a specific problem. Instead of writing AES encryption from scratch (which is hard to get right and dangerous to mess up), we use a library that has been written and audited by experts.

Node.js libraries live in a folder called `node_modules/` and are listed in `package.json`. There are over a million publicly available packages on the npm registry. IPO Manager uses about a dozen directly.

Each library you add brings:

- Functionality you don't have to write yourself.
- A potential security risk — you are trusting the library's authors.
- A maintenance cost — libraries get updates, sometimes with breaking changes.

Picking libraries carefully is a real skill. For this project, every library was chosen for being either widely-used (Electron, React, Playwright) or domain-essential (Argon2, SQLCipher).

Part 3 — How the App Works End-to-End

3.1 The journey of your master password (unlock flow)

Step by step, what happens when you launch the app and type your password:

1. You double-click 'IPO Manager.exe' on your desktop.
2. Windows OS launches the Electron process. Electron starts:
 - a. The main process (a Node.js runtime)
 - b. The renderer process (a Chromium browser window)
3. The renderer loads index.html, which boots the React app.
It calls `window.api.vault.status()` to check if a vault exists.
4. The main process replies:
 - If `vault.meta.json` exists in `%APPDATA%\ipo-manager\data\`:
 `status = initialized=true`, so show the Unlock screen
 - Otherwise: `status = initialized=false`, show first-time setup
5. You type your master password and click Unlock.
6. The renderer sends 'vault:unlock' IPC message to the main process, with the password as the payload.
7. Main process calls `deriveMasterKey(password)`:
 - a. Read salt from `vault.meta.json`
 - b. Run `Argon2id(password, salt, memoryCost=256MB, timeCost=4)`
 - c. Result: 32-byte raw keyThis takes ~500 ms – intentionally slow to defeat brute-force.
8. Main process calls `openDb(key)`:
 - a. Create SQLite connection to `vault.db`
 - b. Set the SQLCipher key via PRAGMA
 - c. Run a test SELECT to verify the key works
 - d. If it fails: throw `INVALID_MASTER_PASSWORD`; the renderer shows the error and waits for another password attempt
 - e. If it succeeds: the DB is open and ready
9. The main process schedules an auto-backup 10 seconds later (giving you time to start interacting before the backup steals the disk).
10. The main process returns `{ ok: true }` to the renderer.
11. The renderer transitions to the Splashing state. The Dashboard component mounts (invisible, behind the splash). It starts loading families, members, Gmail status, CAPTCHA status – all in parallel.
12. SplashScreen plays the 3-note D-major arpeggio via Web Audio API, rotates two gold rings around the IPO mark, fills the loading bar over 1.8 seconds. Then fades out.
13. When the splash fades, the Dashboard is already fully loaded.
You see your families. The vault is unlocked.

3.2 The journey of a bank login (e.g. AU Bank for Akshay)

1. You click on 'AU' next to Akshay's name in the dashboard.
2. The button's onClick calls loginBank(akshayId, auBank, familyId).
3. loginBank() sends 'login:bank' IPC to main process.
4. Main process runs runLogin('BANK', akshayId, auBank.id):
 - a. SELECT * FROM bank_accounts WHERE id = ? AND member_id = ?
 - b. Look up the adapter: getBankAdapter('AU') returns auBankAdapter
 - c. Decrypt user_id_enc → 'akshaySharma123'
 - d. Decrypt password_enc → '.....'
 - e. Decrypt customer_id_enc → 'CUSTAU456789'
5. Main process calls launchSession({ profileKey: 'BANK-AU-42' }):
 - a. Build the browser profile path
 - b. Find Chrome.exe on disk
 - c. Launch a real Chromium window with that profile
 - d. Position and size the window
6. auBankAdapter.login(page, credentials, fetchOtp) runs:
 - a. Navigate to https://retail.aubank.in/login
 - b. Type the customer ID (CRN)
 - c. Click 'Proceed'
 - d. Type the password
 - e. Wait for the CAPTCHA image
 - f. Screenshot the CAPTCHA region
 - g. Call solveCaptchaTextWithClaude(imageBytes)
 - Check usage gate: consented? under daily cap? → yes
 - POST to api.anthropic.com/v1/messages with the image
 - Anthropic responds with the 6-character solution
 - Record the call (today's count + token usage)
 - Return the text
 - h. Type the captcha solution
 - i. Click 'Login'
 - j. Wait for OTP screen
 - k. Call fetchOtp() – this watches Gmail for an AU OTP email
 - l. Type the OTP
 - m. Land on the dashboard page
7. If shouldFetchBalance is true:
adapter.fetchBalance(page) reads the Savings and FD balances from the dashboard. Returns 'Savings: ■8,141.37 | Deposit: ■2,00,000.00'.
8. Main process updates bank_accounts.balance with the new string.
9. audit_log gets a row: action=LOGIN_BANK, target=AU, status=SUCCESS, member_id=akshayId.
10. Main process returns { ok: true, balance: '...' } to the renderer.
11. The renderer updates Akshay's AU chip in the UI.

The browser window stays open. You can switch to it manually to do anything else you want – submit an IPO bid, transfer money, whatever.

3.3 The journey of an AU IPO bid (multi-member)

1. You click 'AU IPO' in the All Members header.
2. The dropdown opens. You tick a family checkbox (selects all AU-capable members in that family). You tick another family. You see the count: 'Start (12)'.
3. You click Start. The renderer iterates the selected members one by one.
4. For each member:
 - a. Show the 'Prepare AU Bid' modal – pick the issue from the catalog, enter quantity (lots), pick CUTOFF or LIMIT.
 - b. You click 'Open AU & Prepare'.
 - c. Renderer sends 'ipo:prepareAuBid' IPC.
 - d. Main process runs the full AU login flow (steps 6 above).
 - e. Once logged in, the adapter navigates to the IPO page, picks the issue, fills the bid, screenshots the page.
 - f. Returns { readyToSubmit, blockedAmount, warnings }.
 - g. The 'Review' modal opens with the screenshot and details.
 - h. You click 'Confirm & Submit'.
 - i. Renderer sends 'ipo:confirmAuBid' IPC.
 - j. Main process clicks 'Submit' on the AU page, captures the bank reference number, records the bid in ipo_bid_runs.
 - k. Closes the browser context for this member.
5. After every member is done, the modal closes. The dashboard shows the updated bid records.

At no point does the app submit a bid without you clicking 'Confirm & Submit'. The 'Open AU & Prepare' step is reversible – 'Cancel' just closes the browser without recording anything.

Part 4 — The Project Folder Structure

Open the folder `H:\ipo-manager_1\ipo-manager\` in Windows Explorer. You will see something like this:

```
ipo-manager/
■■■■ .git/                (hidden – version control internals)
■■■■ .gitignore
■■■■ dist/                (build output – installer goes here)
■■■■ docs/                (project documentation)
■■■■ node_modules/        (downloaded libraries – ~1 GB)
■■■■ out/                 (compiled JS – produced by electron-vite)
■■■■ scripts/             (utility scripts like this PDF generator)
■■■■ src/                 (THE SOURCE CODE – most important folder)
■   ■■■■ main/            (backend – Node.js side)
■   ■■■■ preload/         (bridge between main and renderer)
■   ■■■■ renderer/        (frontend – React UI)
■■■■ tests/              (automated test files)
■■■■ electron.vite.config.ts (build config)
■■■■ LEARNING-GUIDE.pdf    (this file)
■■■■ package.json          (project manifest)
■■■■ package-lock.json     (exact library versions)
■■■■ PROGRESS.md           (development changelog)
■■■■ README.md             (one-page project summary)
■■■■ tsconfig.json         (TypeScript compiler config)
```

Let me explain each folder and file in detail. We will visit them in the order you would naturally encounter them — most important first.

4.1 package.json — the project manifest

This is the single most important configuration file. It declares:

- The name and version of the project ("ipo-manager", "0.1.0").
- The list of **dependencies** — libraries the running app needs.
- The list of **devDependencies** — libraries needed only to build/test.
- Scripts that can be run with `npm run X`.
- The **build** configuration for electron-builder (installer settings).

Here are the actual scripts defined:

```
"scripts": {
  "dev": "electron-vite dev",
  "build": "electron-vite build",
  "build:win": "electron-vite build && electron-builder --win",
  "test": "vitest run",
  "test:watch": "vitest",
  "postinstall": "electron-builder install-app-deps && playwright install chromium"
}
```

When you ran `npm install` for the first time, the `postinstall` script automatically downloaded Playwright's Chromium binary and rebuilt all native modules for your Electron version. That is why a fresh clone takes ~5 minutes — it is downloading hundreds of megabytes of dependencies.

4.2 src/ — the source code

This is the only folder you will ever edit by hand. Everything else is either generated, configuration, or downloaded.

4.2.1 src/main/ — the backend (Node.js side)

This folder contains the code that runs in the Main process — the part of Electron with full access to the file system, network, OS, and native modules.

```
src/main/
■■■ index.ts                (entry point – boots the app, creates the window)
■■■ activity.ts            (auto-lock timer logic – 30 min idle = lock)
■■■ ipc.ts                 (IPC handlers – 1500 lines of message routing)
■■■ logging.ts             (writes automation.log)
■■■ ai/
■   ■■■ anthropic.ts       (Claude API wrapper for CAPTCHA solving)
■   ■■■ captcha.ts         (provider abstraction)
■   ■■■ usage.ts           (daily cap + consent tracking)
■■■ automation/
■   ■■■ browser.ts         (Playwright launcher & shared helpers)
■   ■■■ registry.ts        (maps bank/broker codes to adapters)
■   ■■■ auBank.ts          (AU Bank login + IPO adapter)
■   ■■■ sbiBank.ts         (SBI login + balance fetch)
■   ■■■ yesBank.ts         (SBI login + balance fetch)
■   ■■■ kotakBank.ts       (SBI login + balance fetch)
■   ■■■ iciciBank.ts       (SBI login + balance fetch)
■   ■■■ bobBank.ts         (SBI login + balance fetch)
■   ■■■ pnbBank.ts         (SBI login + balance fetch)
■   ■■■ hdfcBank.ts        (SBI login + balance fetch)
■   ■■■ axisBank.ts        (SBI login + balance fetch)
■   ■■■ genericBank.ts     (fallback template)
■   ■■■ zerodha.ts         (Zerodha broker adapter)
■   ■■■ dhan.ts            (Zerodha broker adapter)
■   ■■■ angel.ts           (Zerodha broker adapter)
■   ■■■ mirae.ts           (Zerodha broker adapter)
■   ■■■ shoonya.ts         (Zerodha broker adapter)
■   ■■■ fyers.ts           (Zerodha broker adapter)
■   ■■■ groww.ts           (Zerodha broker adapter)
■■■ backup/
■   ■■■ engine.ts          (encrypted incremental backup engine)
■■■ crypto/
■   ■■■ master.ts          (Argon2id key derivation from master password)
■   ■■■ field.ts           (AES-256-GCM field encryption)
■■■ db/
■   ■■■ connection.ts      (opens SQLCipher database, runs migrations)
■   ■■■ schema.sql         (table definitions – for humans to read)
■   ■■■ schema.ts          (table definitions – inlined for runtime)
■■■ documents/
■   ■■■ storage.ts         (encrypted PDF/JPEG storage)
■■■ email/
■   ■■■ gmail.ts           (Gmail OAuth + OTP fetcher)
■■■ exporter/
■   ■■■ excel.ts           (exports vault to Excel for backup)
■■■ importer/
■   ■■■ excel.ts           (parses Demat_Sheet.xlsx for first-time load)
■■■ ipo/
■   ■■■ catalog.ts         (BSE scraper for IPO master list)
■■■ reports/
```

```
■■■ storage.ts
■■■ angelWorkbook.ts      (parsing Angel Excel portfolio report)
■■■ dhanWorkbook.ts
■■■ zerodhaWorkbook.ts
```

Each file has a specific job. Let me explain the most important ones in detail.

index.ts — the entry point

This is what runs first when Electron starts. It does five things:

- Creates the BrowserWindow (1280 × 820 pixels, dark background).
- Removes the default menu bar (cleaner look).
- Tells Electron to show the window only after it is ready to render (no white flash).
- Registers all the IPC handlers from ipc.ts.
- Starts the auto-lock timer (checks every 30 seconds if the user has been idle 30+ minutes).

ipc.ts — the message router

Most of the backend logic lives here. It is a giant file (about 1500 lines) that registers every IPC handler the renderer can call. Each handler looks like:

```
ipc.handle('families:list', () => {
  const db = getDb();
  return db.prepare('SELECT * FROM families ORDER BY display_order').all();
});
```

That handler responds to messages with the name 'families:list'. It reads from the database and returns the rows. The renderer calls it like:

```
const families = await window.api.families.list();
```

In a future refactor we should split ipc.ts by domain — ipc/vault.ts, ipc/families.ts, ipc/login.ts, etc. — to make the file easier to navigate. For now it is one big file.

crypto/master.ts — turning a password into a key

When you type your master password, this file's deriveMasterKey() function runs the Argon2id algorithm to produce a 32-byte key. The key, not the password, is then handed to SQLCipher. Argon2id is designed to be:

- **Memory-hard** — needs 256 MB of RAM to compute, which makes it impractical for GPU-based brute-force attacks (GPUs have lots of cores but limited RAM per core).
- **Time-hard** — takes ~500 ms per try on a modern CPU. An attacker testing 1 million passwords needs 5.7 days (vs. milliseconds for simpler KDFs).
- **Deterministic** — same password + same salt = same key. Without the salt (which is stored in vault.meta.json), the same password would give a different key on every machine. The salt is not secret — it is stored next to the vault.

crypto/field.ts — encrypting individual fields

Even with SQLCipher protecting the whole database file, we add a second layer: each sensitive value (password, PAN, Aadhaar, account number) is also encrypted with AES-256-GCM using a separate key stored in the OS keychain. This is defense-in-depth: an attacker who somehow gets the DB file *and* guesses the master password still cannot read the credentials without OS-level access.

db/connection.ts — opening the database

Opens the SQLite file, sets the SQLCipher key, runs any pending migrations (idempotent schema upgrades — e.g. "if the email_password_enc column doesn't exist, ALTER TABLE to add it"). The migrations let an old database work with a newer version of the app without losing data.

automation/browser.ts — launching Playwright

This file handles everything about the real browser windows we open. It:

- Finds chrome.exe or msedge.exe on the user's machine (system Program Files OR per-user LocalAppData).
- Creates a persistent profile directory per (member, bank/broker) so cookies are remembered between logins.
- Configures the browser to disable safebrowsing download warnings (so the automation can save broker reports without a popup).
- Provides shared helpers used by every adapter — `getMostRecentOpenPage`, `resolveBrowserDownload`, etc.
- Provides `closeAllBrowserSessions()` and `purgeBrowserProfiles()` for the lock-time cleanup.

automation/registry.ts — the adapter index

A tiny file that maps bank/broker codes to adapter implementations:

```
const BANK_ADAPTERS = {
  AU:    auBankAdapter,
  YES:   yesBankAdapter,
  SBI:   sbiBankAdapter,
  // ...
};

export function getBankAdapter(code) {
  return BANK_ADAPTERS[code] || null;
}
```

automation/auBank.ts — the most complex adapter

AU Bank gets the most attention because it is the bank we use to file IPO bids. It includes:

- `login()` — the full multi-step login flow (CRN → password → CAPTCHA → OTP → dashboard)
- `fetchBalance()` — scrapes Savings + FD from the dashboard
- `prepareIpoBid()` — navigates to the IPO page, fills the bid, screenshots for review
- `submitPreparedIpoBid()` — clicks final Submit and captures the bank reference

backup/engine.ts — the backup system

Built recently to solve the catastrophic-data-loss risk. Implements:

- `createSnapshot()` — produces a content-addressed incremental snapshot
- `listSnapshots()` — groups them by Last 24h / 7d / 30d / 6mo retention bands
- `restoreSnapshot()` — re-derives the key from the snapshot's own salt (critical for cross-machine restore)
- `pruneOldSnapshots()` — applies the retention policy
- `garbageCollectBlobs()` — deletes documents no surviving snapshot references

4.2.2 src/renderer/ — the frontend (React UI)

```
src/renderer/  
■■■ index.html           (the HTML shell – almost empty, React fills it in)  
■■■ src/  
    ■■■ main.tsx         (React entry point – calls root.render(<App />))  
    ■■■ App.tsx          (top-level state machine: loading → unlock → unlocked)  
    ■■■ styles.css       (3000+ lines of CSS – the entire visual design)  
    ■■■ assets/  
        ■■■ logos/      (PNG/SVG logos for HDFC, ANGEL, PNB, etc.)  
    ■■■ pages/  
        ■■■ Unlock.tsx   (the master password screen)  
        ■■■ SplashScreen.tsx (3-note unlock sound + revolving gold rings)  
        ■■■ Dashboard.tsx (4200 lines – almost all the UI lives here)
```

Notice the asymmetry: the main process has ~30 files; the renderer has just 4 real component files. That is because `Dashboard.tsx` is a giant monolith that needs to be split into smaller pieces in a future refactor. For now, it works.

App.tsx — the top-level state machine

This is the React component that controls which screen the user sees. It manages four states:

- **loading** — initial state, shows "Loading..." while checking vault status.
- **unlock** — shows the password screen. Subdivided into `firstTime` / `not-firstTime`.
- **splashing** — vault just unlocked; show the splash overlay while Dashboard loads data in the background.
- **unlocked** — Dashboard is fully visible; the splash is gone.

Dashboard.tsx — almost everything

This single file contains: the sidebar, the All Members view, the single-family view, the Spreadsheet view, every modal (member edit, AU bid prep, AU bid review, service config, backup settings, restore, member detail card, change password), the bulk action handlers, the toast notification system, the drag-and-drop reorder logic, the IPC call wrappers, and many small utility functions.

4200 lines in one file is too many. The next refactor should split this into:

- `components/Sidebar.tsx`
- `components/FamilyAccordion.tsx`

- components/MemberRow.tsx
- components/BackupSettingsModal.tsx
- components/MemberDetailCard.tsx
- components/SpreadsheetView.tsx
- hooks/useBackupStatus.ts, useCaptchaUsage.ts, useGmailStatus.ts

But it is not a priority right now — the file works and changes to it are rare. We will revisit this when adding the auto-bid scheduler feature.

4.2.3 src/preload/ — the bridge

src/preload/
 ■■■ index.ts (about 100 lines — defines window.api)

The preload script is the contract between the main and renderer processes. It is the *only* code that can call `ipcRenderer.invoke()` from the renderer. Without preload, the renderer (running in a sandboxed Chromium tab) would have no way to talk to the main process.

Every IPC call the renderer makes goes through `window.api`, which is defined here. For example:

```
const api = {
  vault: {
    status: () => ipcRenderer.invoke('vault:status'),
    unlock: (pw) => ipcRenderer.invoke('vault:unlock', pw),
    lock:    () => ipcRenderer.invoke('vault:lock'),
    // ...
  },
  families: {
    list:    () => ipcRenderer.invoke('families:list'),
    create: (name, min) => ipcRenderer.invoke('families:create', { family_name: name, min_balance: min }),
    // ...
  },
  // ... and so on for backup, login, captchaAi, gmail, ipo, etc.
};

contextBridge.exposeInMainWorld('api', api);
```

This bridge is a deliberately narrow surface. The renderer can *only* call the methods listed in `window.api`. It cannot, for instance, read arbitrary files or run arbitrary IPC channels. This is a security boundary.

4.3 node_modules/ — the downloaded libraries

After running `npm install`, you will see a `node_modules/` folder roughly 1 GB in size, containing hundreds of subdirectories. Each subdirectory is a downloaded library. You almost never look inside this folder — it is managed automatically.

Think of it like a vendor's warehouse you don't catalogue yourself. The `package-lock.json` file is the inventory list — it records the exact version of every library so that the next person who runs `npm install` gets identical bytes.

4.4 dist/ — the build output

When you run `npm run build:win`, the installer goes here. After a successful build you will see:

```

dist/
■■■ win-unpacked/                                (the app, unpacked)
■   ■■■ IPO Manager.exe                          (the actual program)
■   ■■■ resources/
■   ■   ■■■ app.asar                             (your compiled code, packed)
■   ■   ■■■ app.asar.unpacked/                  (native modules that can't be in .asar)
■   ■■■ ffmpeg.dll, vk_swiftshader.dll, ...    (Chromium support DLLs)
■   ■■■ locales/                               (Chromium translations)
■■■ IPO-Manager-Setup-0.1.0-x64.exe             (THE INSTALLER – share this file)
■■■ IPO-Manager-Setup-0.1.0-x64.exe.blockmap
■■■ builder-effective-config.yaml

```

The single file you share with someone else is `IPO-Manager-Setup-0.1.0-x64.exe` (about 100 MB). Everything else is build scaffolding.

4.5 out/ — the compiled JavaScript

When the build runs, electron-vite transforms every `.ts` file in `src/` into a `.js` file in `out/`. This is a *technical* step that you almost never need to look at. The compiled code is what Electron actually executes.

```

out/
■■■ main/
■   ■■■ index.js      (all of src/main/ combined into one ~400 KB file)
■■■ preload/
■   ■■■ index.js      (~5 KB)
■■■ renderer/
    ■■■ index.html
    ■■■ assets/
        ■■■ index-XXXX.js  (the React app, ~450 KB)
        ■■■ index-XXXX.css (the compiled styles, ~65 KB)
        ■■■ *.png, *.svg   (the bank/broker logos)

```

4.6 tests/ — automated tests

```

tests/
■■■ _stubs/
■   ■■■ keytar.ts      (in-memory keychain stub for tests)
■■■ backup/
■   ■■■ engine.test.ts (cross-salt restore – conditionally skipped)
■■■ crypto/
    ■■■ master.test.ts (Argon2 round-trip + salt sensitivity)
    ■■■ field.test.ts  (AES-GCM round-trip + tamper detection)

```

Right now there are 18 tests passing + 4 conditionally skipped. The skipped ones need Electron's Node ABI to load the SQLite native module. The tests run with `npm test` in about 7 seconds.

4.7 Configuration files

tsconfig.json

Tells the TypeScript compiler how to behave: which version of JavaScript to emit, where to find type definitions, which directories to compile. You almost never need to edit this file.

electron.vite.config.ts

Tells electron-vite (the build tool) how to build the three parts (main, preload, renderer). Each part gets its own build pipeline because they have different requirements (renderer needs HTML+CSS+JS bundling; main needs Node-compatible output).

.gitignore

Lists files and folders that should NOT be tracked in version control: node_modules/, dist/, out/, vault.db, etc. This keeps the repo small and prevents committing sensitive data by accident.

Part 5 — Every Library, Explained

This Part walks through each library the app depends on. For each one I will tell you what it does, why it is here, what would break if it were removed, and what the alternatives are.

5.1 electron — the desktop app framework

What it does: bundles Chromium + Node.js into a single distributable app. Provides the `BrowserWindow` class to create OS-native windows, the `ipcMain` and `ipcRenderer` modules for cross-process messaging, the `app` and `dialog` modules for OS integration.

Why this app needs it: without Electron we couldn't run JavaScript code with file-system access on Windows. The alternative would be writing a separate Windows app in C#/WPF, plus a separate Mac app in Swift — three times the work.

Alternatives: Tauri (uses native webview, smaller bundle), Wails (Go-based), Neutralino (very lightweight). For our use case Electron's maturity wins.

5.2 react + react-dom — the UI framework

What it does: a JavaScript library for building component-based user interfaces. Lets you describe the UI as a function of state, and re-renders automatically when state changes.

Why this app needs it: the dashboard has lots of interactive state — selected family, expanded accordion rows, open modal, edit mode toggle, busy spinners. React makes managing this state predictable.

Alternatives: Vue, Svelte, Solid. All work; React was picked for familiarity and ecosystem size.

5.3 typescript — the type-safety layer

What it does: adds optional type annotations to JavaScript, catches type-related errors at build time before the program runs.

Why this app needs it: a financial app cannot afford runtime type surprises. "Did you mean to pass a number where a string was expected?" should be caught at the compiler, not by a confused user.

Alternatives: Flow (Facebook's TS competitor — losing ground), JSDoc-typed JavaScript (less ergonomic).

5.4 playwright — the browser automation engine

What it does: launches and controls a real web browser (Chromium, Firefox, or WebKit) from your code. You can fill forms, click buttons, take screenshots, download files, intercept network requests, and so on.

Why this app needs it: banks and brokers do not have programmer-friendly APIs we can call. To check a balance or submit an IPO bid, we have to drive their website like a human would. Playwright lets us automate that reliably.

Alternatives: Puppeteer (Google's predecessor — same author moved to Playwright), Selenium (older, slower, more fragile). Playwright is the modern standard.

Why we use a real Chrome instead of a headless one: banks deploy bot-detection. A real visible Chromium with normal user-agent and a real profile directory is much less likely to be flagged than a headless browser. It also lets you intervene manually if the automation hits an unexpected screen.

5.5 better-sqlite3-multiple-ciphers — encrypted SQLite

What it does: a Node.js binding to SQLite — a tiny, file-based database. The 'multiple-ciphers' variant adds SQLCipher: every page of the database file is encrypted with AES-256 before being written to disk.

Why this app needs it: we need a database (lots of structured queries on families, members, bank accounts), it must be local (no server), and the file must be useless without the master password. SQLite + SQLCipher checks all three boxes.

Alternatives: raw SQLite + manual file encryption (more work, more bugs); a server-based DB like PostgreSQL (defeats the local-only design); a JSON file (can't handle complex queries efficiently). SQLite + SQLCipher is the right tool.

5.6 argon2 — the password hashing library

What it does: implements the Argon2id algorithm — the modern best-practice for turning a user password into a cryptographic key.

Why this app needs it: we cannot use the password directly as the encryption key (passwords are short and predictable; keys must be long and uniform). Argon2id is intentionally slow and memory-heavy, making brute-force attacks prohibitively expensive.

Alternatives: bcrypt (older, GPU-attackable), scrypt (similar to Argon2 but less standardised), PBKDF2 (much weaker against GPUs). Argon2id won the Password Hashing Competition in 2015 and is the consensus modern choice.

5.7 keytar — the OS keychain bridge

What it does: stores and retrieves passwords/keys from the operating system's secure credential store — Windows Credential Manager on Windows, Keychain on macOS, libsecret on Linux.

Why this app needs it: the per-field encryption key has to be stored *somewhere*. Storing it in a plain file would defeat the purpose. The OS keychain is the right place — protected by the user's OS login, isolated from other apps.

Three entries this app creates:

- ipo-manager / field-encryption-key-v1 — the 32-byte field encryption key (random per-machine).
- ipo-manager / gmail-refresh-token-v1 — the OAuth refresh token for Gmail.
- ipo-manager / anthropic-api-key-v1 — your Anthropic API key for CAPTCHA solving.

You can see all three in Windows: Control Panel → Credential Manager → Windows Credentials → ipo-manager.

5.8 googleapis — the Gmail API client

What it does: Google's official Node.js client for all their APIs. We use only the Gmail subset to watch for OTP emails.

Why this app needs it: when a bank sends an OTP to your email, we need to read it programmatically. Google's API provides a clean way: search for messages matching a query, extract the body, regex out the 6-digit code.

OAuth flow: first time you connect Gmail, the app opens your browser, you sign in to Google, Google sends back a refresh token. We store that token in keytar. From then on, the token is used to issue short-lived access tokens automatically — you never need to sign in again unless you revoke access.

5.9 electron-vite + vite + electron-builder — the build chain

vite: the underlying build tool. Compiles TypeScript, bundles JavaScript modules, transforms CSS, ships an HTML index.

electron-vite: a thin wrapper that knows about the three-part structure of an Electron app (main / preload / renderer). Runs vite three times with different configs.

electron-builder: takes the compiled out/ directory plus your node_modules and packages it into the .exe installer. Handles code signing, auto-updater integration, NSIS installer scripting.

5.10 Specialty libraries

xlsx (SheetJS)

Reads and writes Excel spreadsheets. We use it for the initial import of the Demat_Sheet.xlsx and for exporting a plain Excel copy of the vault.

tesseract.js

Optical character recognition (OCR) — extracts text from images. Bundled but not currently used; was originally an alternative to Anthropic for CAPTCHA solving.

pngjs

Reads and writes PNG image files in pure JavaScript. Used for screenshot processing before sending to the CAPTCHA solver.

totp-generator

Generates 6-digit TOTP (Time-based One-Time Password) codes from a base32 secret — the same algorithm Google Authenticator uses. Used for brokers that require TOTP login.

5.11 vitest — the test runner

What it does: runs the files in tests/ and reports which pass and which fail.

Why we use it: vitest is the fastest popular test runner for TypeScript projects. It reuses our vite config, so tests have the same module resolution as production code.

Part 6 — The Security Model

This Part is the most important for any user storing 30+ relatives' credentials. Read it carefully.

6.1 Defense in depth

Security people use a phrase: *defense in depth*. It means: don't rely on one wall. Build many walls, each of which would need to be breached for the attacker to succeed. IPO Manager has four walls:

- **Wall 1 — the OS user account.** The vault file lives in your Windows user profile. Another user on the same PC cannot see it (unless they are an administrator).
- **Wall 2 — SQLCipher.** The entire database file is encrypted with a key derived from your master password via Argon2id. An attacker with the file but no password cannot read a single row.
- **Wall 3 — field-level encryption.** Sensitive columns (PAN, Aadhaar, passwords) are encrypted a second time with a random 32-byte key stored in the OS keychain. An attacker who has the DB file AND has guessed the master password still cannot read credentials without keychain access.
- **Wall 4 — the OS keychain.** The field encryption key is protected by the OS's own credential store. On Windows, that is Credential Manager, protected by your Windows login.

To read your stored bank password, an attacker would need to bypass all four walls. The realistic ways that could happen are surprisingly limited (covered in 6.4).

6.2 The three encryption layers

Layer 1: **HTTPS** — every connection to a bank, broker, Gmail, or Anthropic is encrypted in transit by the browser. This is standard everywhere; we get it for free.

Layer 2: **SQLCipher (AES-256-CBC + HMAC-SHA512)** — encrypts the entire vault.db file at rest. Key = Argon2id(master_password, salt).

Layer 3: **Field encryption (AES-256-GCM)** — encrypts individual sensitive columns within the database. Key = random 32 bytes in keychain.

6.3 Where each key lives

Key	Derived from	Stored at	Used to
Master key (32 bytes)	Argon2id(password, salt)	Never stored — in memory only while vault is open	Operates the SQLCipher DB
Field key (32 bytes)	Random when first generated	OS keychain (keytar)	AES-GCM encrypt/decrypt sensitive columns
Argon2 salt (16 bytes)	Random when vault first created	vault.meta.json (plaintext)	Reproduce the master key from the password
Backup field-key.bin	Field key AES-GCM-encrypted with master key	Inside master key's hot folder	Restore the field key on another machine

6.4 Threat model — who are we protecting against?

"Threat model" is the question: *what kinds of attackers do we worry about, and what can each one realistically do?* Listed from weakest to strongest:

Threat 1: Casual snoop with brief physical access

Someone glances at your unlocked PC for 30 seconds while you are at lunch. What they can see depends on whether the vault is locked:

- **If locked:** nothing — they see the unlock screen.
- **If unlocked:** anything currently on screen — the family list, the active modals. They could click around the UI but every credential is either masked (passwords show as bullets) or only revealed by clicking (detail card cells). They cannot copy out the vault file in 30 seconds.

Mitigation: Ctrl+L locks immediately. Auto-lock after 30 minutes idle. Train yourself to hit Ctrl+L when stepping away.

Threat 2: Persistent local user (e.g. office IT, family member)

Someone with the same Windows account, or admin rights on your PC. They have hours of access. Without the master password they:

- Can find vault.db in %APPDATA% (yes — it's just a file).
- Cannot read it (SQLCipher).
- Could try to brute-force the master password — but Argon2id makes this 5+ days per million tries on a decent CPU; a strong password is effectively safe.
- **Can read browser session cookies** in the browser-profiles folder — this would let them replay a bank login. **This is the most serious risk**, mitigated by the lock-time browser session purge.

Threat 3: Stolen disk (lost laptop, sold hard drive)

Worst-case offline attack: an attacker has the entire disk image, all the time in the world. With a strong master password and Argon2id parameters, this is effectively safe — but no encryption survives a guess of "password" or "akshay123". **The master password is the weakest link.** Use 16+ characters with mixed case, digits, symbols, and no dictionary words.

Threat 4: Malware running as you

If a virus is running as your Windows user, it has the same rights you do — including access to the OS keychain. Nothing the app can do prevents this. Mitigations: keep Windows Defender on, do not install random .exe files, do not click email attachments from strangers.

Threat 5: A targeted attack by a nation-state

Outside the design scope of this app. If a state actor wants your data, they will get it through other means (compromising the Anthropic API endpoint, intercepting OTPs at the carrier, etc.).

6.5 What we DO protect against

- Theft of the laptop / disk → AES-256 encryption is unbroken in any remotely-practical time.
- Casual access during a lunch break → manual lock, auto-lock, password masking in UI.
- Cloud-account compromise → there is no cloud account; data never leaves the local PC except for the user-chosen backup folder.
- Credential leak from a third-party service → no third-party service has your credentials.

6.6 What we DON'T protect against

- Weak master password (this is on you).
- Malware running as you (this is on Windows Defender).
- A coerced unlock (someone forcing you to type the password — encryption fundamentally can't help here).
- A breach of Anthropic or Google (we send some metadata to both — see 6.7).

6.7 What leaves the machine — explicitly listed

Things that go off-disk in normal use:

- **To bank/broker websites:** your credentials. This is the entire point.
- **To Gmail:** the queries to read OTP emails. Gmail already had those emails — we just read them.
- **To Anthropic (api.anthropic.com):** the AU Bank CAPTCHA image, ONLY when AU login is in progress AND consent is recorded AND daily cap not reached. ~4 KB per image.
- **To BSE / NSE:** requests for the public IPO catalog. No personally-identifiable data.
- **To your chosen backup folder (OneDrive/Drive/Dropbox if used):** encrypted snapshots. The cloud provider sees encrypted blobs, not your credentials.

Things that **never** leave the machine:

- Your master password (it is hashed locally, never transmitted).
- The decrypted credentials themselves.
- The audit log.
- Anything in vault.db.

Part 7 — Features: What Works

7.1 Vault setup and unlock

First-time setup: on the first launch, the app asks you to create a master password. You type it, confirm it, click "Create vault". The app:

- Validates password strength (≥ 12 chars, mixed case, digit, symbol, no common words).
- Generates a random 16-byte salt.
- Writes vault.meta.json.
- Derives the master key (Argon2id, ~ 500 ms).
- Creates vault.db with the schema embedded in schema.ts.
- Generates a random 32-byte field key, stores it in OS keychain.
- Transitions to the dashboard (empty — no families yet).

Subsequent unlocks: type the password, hit Unlock, see the 1.8-second splash, land on your data.

7.2 Family and member management

Click "+ Family" in the sidebar to add a family. Set a minimum-balance threshold (for the Balance Management dashboard).

Click "+ Member" inside a family to add a member. Enter name, PAN, Aadhaar, DOB, mobile, email, email password. Add bank accounts (user-id, password, customer-id, account number, IFSC). Add broker accounts (user-id, password, client-id, TOTP secret if used). Upload document softcopies (PAN PDF, Aadhaar, cancelled cheque).

Drag-and-drop reorder works both at family level (sidebar) and member level (within a family).

7.3 Bank login automation

Per bank, the app implements the full login flow: navigate to the bank's login page, fill credentials, fetch OTP from Gmail (or use TOTP), solve CAPTCHA via Claude (AU Bank only), land on the dashboard, scrape the balance, update the DB.

Supported banks (9): AU, YES, SBI, KOTAK, ICICI, BOB, PNB, HDFC, AXIS.

The browser window stays open after login so you can do anything else manually if you want.

7.4 Broker login + portfolio fetch

Same pattern for brokers (7): Zerodha, Dhan, Angel One, Mirae, Shoonya, Fyers, Groww.

For Zerodha / Dhan / Angel, the app can also download the holdings Excel report and parse it into structured rows (broker_portfolio_reports + broker_portfolio_holdings tables). The parsed data drives the portfolio view modal.

7.5 AU Bank IPO bidding

This is the headline feature. From the All Members header → AU IPO dropdown:

- Tick individual members or whole families via the indeterminate-checkbox.
- Pick an issue from the BSE catalog (cached locally, refreshed on demand).
- Enter quantity (in lots) and bid type (CUTOFF or LIMIT).
- Click Open AU & Prepare — the app logs into AU for each member in sequence, fills the bid, screenshots it for review.
- Confirm or Cancel each one. Only Confirmed bids are actually submitted.
- Each submitted bid is recorded in ipo_bid_runs with bank reference number.

7.6 Excel import / export

Import: parse a Demat_Sheet.xlsx where each sheet is one family and each column is one member. Used for the initial bulk load from your existing records. Schema documented inline in src/main/importer/excel.ts.

Export: produce a plaintext Excel copy of the vault for human review or external backup. Requires the master password to be currently in memory (i.e. vault unlocked).

7.7 Encrypted incremental backup

Choose a backup folder (ideally inside OneDrive/Drive for multi-machine sync). The engine:

- Creates a snapshot 10s after unlock and every 6h thereafter.
- Each snapshot includes a consistent SQLCipher-encrypted DB copy (via VACUUM INTO), vault.meta.json (salt for cross-machine restore), and field-key.bin (field key encrypted with the master key).
- Document files (PDFs/JPEGs) live in a shared content-addressed blobs/ folder — same file referenced from many snapshots is stored ONCE.
- Retention bands: keep ALL in last 24h, ONE per day in last 7d, ONE per week in last 30d, ONE per month in last 6mo.
- Garbage collection removes blobs no surviving snapshot references.

Restore: pick a snapshot, type the master password. Engine re-derives the key from the snapshot's own salt — so the same password works across machines even though they have different local salts.

7.8 Member detail card (the credentials cockpit)

Click any member's name → modal opens with compact tables:

- Identity (Name / PAN / Aadhaar / DOB / Mobile / Email / Email Password)
- Banks (one row per configured bank: User ID / Password / Customer ID / Account No. / IFSC)
- Brokers (one row per configured broker: User ID / Password / Client ID / TOTP / Mobile / Email)

Click any cell → copies the raw value to the clipboard. Secrets are masked in the display (•••••) but the clipboard gets the real text.

7.9 Spreadsheet view

New top-level view in the sidebar nav: a flat sortable, filterable grid of all members × all banks/brokers. Rows = members, columns = each configured bank's balance + each configured broker's portfolio value + totals.

Search by name/family/mobile/email. Filter by family. Sort by any column. Click a member name → opens the detail card.

7.10 Cost-safety guardrails for CAPTCHA solving

- **Daily call cap** (default 100) — refuses calls past the cap with a clear log line.
- **Consent flag** — must be ticked before any CAPTCHA image is uploaded (auto-flipped when you save the Anthropic key).
- **Per-day token tracking** — input + output token counts.
- **Sidebar pill** shows today's count: e.g. CAPTCHA AI: Claude ready (3/100)

7.11 Manual lock + Ctrl+L

■ Lock button in the sidebar header. Ctrl+L global shortcut. Locks the DB, wipes in-memory secrets, purges browser sessions, returns to unlock screen.

7.12 Factory reset

Backup Settings → Danger zone → "Reset everything..." Type RESET to confirm. Wipes the entire data directory, browser profiles, and keychain entries. Backup folder is untouched.

Part 8 — Limitations and Known Issues

8.1 Selector brittleness — websites change

Every adapter (auBank.ts, sbiBank.ts, etc.) clicks buttons and fills inputs by addressing elements on the bank's webpage using *selectors* — patterns like "the input with placeholder='User ID'" or "the button containing the text 'Login'".

When the bank redesigns their website (which happens every 6–18 months), those selectors break. Symptoms: the automation hits a timeout, or types into the wrong field, or never clicks the right button.

Workaround: open the bank's site manually, inspect the new HTML, update the selectors in the corresponding adapter file. There is no automated way around this — it is an inherent cost of website automation.

8.2 Only AU IPO bidding is automated

The full IPO bid flow (prepare → review → submit) is only implemented for AU Bank. Other banks support login + balance fetch but not IPO bidding. Adding more banks requires writing each bank's `prepareIpoBid` + `submitPreparedIpoBid` functions — non-trivial because every bank's IPO page is different.

8.3 No allotment tracking

After the IPO closes, you have to check allotment status manually (via BSE/NSE/registrar websites). The app does not yet automate this.

8.4 No auto-bid scheduler

You have to be at your desk during the IPO open window to click "Open AU & Prepare". A future feature: schedule bids to fire at 10:01 AM on Day 1.

8.5 No mobile app

Windows desktop only. No mobile or web version. Multi-machine support is via backup folder sync (OneDrive/Drive), not via real-time sync.

8.6 Browser-dependent

Requires Google Chrome or Microsoft Edge installed on the machine. The automation cannot use Firefox in the current configuration. Most Windows users already have Chrome or Edge, so this is rarely a real obstacle.

8.7 No code signing

When the recipient runs the installer on a fresh PC, Windows SmartScreen shows "Windows protected your PC — unrecognized app". They have to click "More info → Run anyway". This is because we are not paying for a code signing certificate (~\$15,000/yr). Functionally harmless but looks intimidating to first-time users.

8.8 No auto-update

To deliver a new build, you have to send the installer file again (or share a download link). The current app does not check for updates on startup. Future: wire up electron-updater.

8.9 Logs grow unbounded

automation.log appends forever, no rotation. After many months it could be hundreds of MB. Plan: rotate at 5 MB or daily, keep last 10 files. Tracked as a yellow audit item.

8.10 No DB indexes on some columns

The `holdings`, `ipo_applications`, `documents.doc_type`, `bank_accounts.bank_code`, and `broker_accounts.broker_code` columns are not indexed. For your current data size (~35 members, ~250 accounts) this is invisible. At 1000+ accounts queries would slow down. Easy fix when needed.

Part 9 — Security Risks — Current Posture

9.1 Critical risks — now mitigated

Risk	Mitigation
Vault file lost or corrupted	Encrypted incremental backup engine with retention bands and multi-machine restore.
Browser cookies replayable	Profile purge on every lock (manual + auto-lock) + manual button.
Anthropic API spend runaway	Daily call cap, token tracking, consent flag, per-call gate.
No way to lock vault on demand	Ctrl+L + sidebar Lock button + auto-lock at 30 min idle.
No tests on security-critical paths	18 vitest tests covering Argon2 round-trip, AES-GCM tamper detection, key sensitivity to

9.2 Remaining risks (yellow, not red)

Risk	Mitigation status / next step
Weak master password	Strength check in master.ts (12+ chars, mixed). Could upgrade to zxcvbn library.
Malware on user's PC	Outside app scope — relies on Windows Defender.
Audit log accumulates forever	Inserts done; UI to view + retention not yet built.
No clipboard auto-clear after copy	Member detail card copies value to clipboard; stays there until you copy something else
No code signing on installer	Cost ~\$15,000/year. Visible only as a SmartScreen warning, no functional impact.
No auto-update	Manual reinstall. Plan: electron-updater with a private S3 release bucket.

9.3 What could go wrong — failure scenarios

Scenario: "I forgot my master password"

There is no recovery. We deliberately do not have a recovery mechanism — that would defeat the encryption. **Mitigation:** write the master password on paper, store it in your office safe.

Scenario: "My laptop hard drive died"

If you had backups configured (Backup Settings → folder inside OneDrive/Drive), you can install the app on a new machine and restore from the cloud-synced snapshot folder. Recovery time: ~15 minutes.

Mitigation: configure backups to a cloud-synced folder.

Scenario: "A bank changes its login flow"

The adapter for that bank breaks. You see a timeout error in the toast. **Workaround:** open the bank manually for now; ping for an adapter update.

Scenario: "My OneDrive backup folder gets deleted"

OneDrive (and Drive, Dropbox) keep deleted files in their Recycle Bin for 30 days. Restore from there.

Mitigation: if super-paranoid, configure TWO backup folders — one cloud, one local on a different physical drive.

Scenario: "Someone steals my laptop"

With a strong master password (16+ chars, no dictionary words), the vault is mathematically safe — they'd need years to brute-force Argon2id. Lock immediately. **Mitigation:** change passwords on the most critical banks within 24 hours anyway, just to be safe (the attacker might extract session cookies before you can wipe the drive remotely).

Part 10 — One Click, End to End

To make all the abstract concepts concrete, let's follow a single user action through every file and function it touches.

The action: you click "Refresh AU" next to Akshay Sharma's name in the All Members view.

Step 1: The click is dispatched in React

In `Dashboard.tsx`, the AU bank chip is rendered inside a member row. The button has an `onClick` handler:

```
<button
  className="account-name-button bank-name-button"
  onClick={() => loginBank(m.id, bank, family.id, { fetchBalance: true })}>
  AU
</button>
```

When you click, React fires the handler. The handler is `loginBank()` — a function defined elsewhere in `Dashboard.tsx`.

Step 2: `loginBank()` prepares the IPC call

```
async function loginBank(memberId, bank, familyId, opts) {
  setBusy(`bank-${bank.id}`); // show spinner on this chip
  showToast('info', `Refreshing ${bank.bank_code}...`);

  try {
    const result = await window.api.login.bank( // <- IPC call to main
      memberId, bank.id, { closeAfterFetch: false }
    );
    if (result.ok) {
      showToast('success', `${bank.bank_code} refreshed`);
      patchBankResult(memberId, bank, familyId, result);
    } else {
      showToast('error', `${bank.bank_code}: ${result.error}`);
    }
  } finally {
    setBusy(null); // hide spinner
  }
}
```

Step 3: The preload bridge

`window.api.login.bank()` is defined in `src/preload/index.ts`:

```
login: {
  bank: (memberId, bankId, options) =>
    ipcRenderer.invoke('login:bank', { memberId, bankId, closeAfterFetch: !!options?.closeAfterFetch })
}
```

This sends an IPC message to the main process with the channel name 'login:bank' and the payload.

Step 4: Main process receives the message

In src/main/ipc.ts:

```
ipc.handle('login:bank', async (_, payload) => {
  return runLogin('BANK', payload.memberId, payload.bankId, {
    fetchBalance: true,
    closeAfterFetch: payload.closeAfterFetch ?? false,
  });
});
```

runLogin is a generic function used for both banks and brokers. Let me walk through what it does.

Step 5: runLogin loads credentials from the database

```
async function runLogin(kind, memberId, accountId, options) {
  const db = getDb();
  const account = db.prepare(
    'SELECT * FROM bank_accounts WHERE id = ? AND member_id = ?'
  ).get(accountId, memberId);

  if (!account) return { ok: false, error: 'Account not found' };

  const code = account.bank_code;          // 'AU'
  const adapter = getBankAdapter(code);    // auBankAdapter
  if (!adapter) return { ok: false, error: `No adapter for ${code}` };

  const username = await decryptField(account.user_id_enc);
  const password = await decryptField(account.password_enc);
  const customerId = await decryptField(account.customer_id_enc);
  // ...
}
```

Step 6: launchSession opens a real Chromium window

```
const { context, page } = await launchSession({
  profileKey: 'BANK-AU-42'                // unique per (kind, code, member)
});
```

If a previous session for this profile is cached, it is reused (cookies still valid → skips the login). Otherwise launches a fresh Chromium instance with a persistent profile.

Step 7: The adapter runs the actual login

In src/main/automation/auBank.ts:

```
async function login(page, creds, fetchOtp) {
  await page.goto('https://retail.aubank.in/...', { waitUntil: 'domcontentloaded' });
  await page.fill('input[name="customerid"]', creds.customerId);
  await page.click('button:has-text("Proceed")');
  await page.fill('input[type="password"]', creds.password);

  // Solve CAPTCHA
  const captchaImg = await page.locator('img.captcha').screenshot();
  const captchaText = await solveCaptchaTextWithClaude(captchaImg);
  await page.fill('input[name="captcha"]', captchaText);

  await page.click('button:has-text("Login")');
```

```

// Wait for OTP screen, fetch OTP from Gmail
await page.waitForSelector('input[name="otp"]');
const otp = await fetchOtp();
await page.fill('input[name="otp"]', otp);
await page.click('button:has-text("Submit")');

// Wait for dashboard
await page.waitForURL(/dashboard/, { timeout: 30000 });
}

```

Step 8: Balance is fetched

```

const balance = await adapter.fetchBalance(page);
// e.g. 'Savings: ■8,141.37 | Deposit: ■2,00,000.00'

```

```

db.prepare('UPDATE bank_accounts SET balance = ?, balance_fetched_at = CURRENT_TIMESTAMP WHERE id = ?')
  .run(balance, accountId);

```

Step 9: Audit log is written

```

auditInsert.run(memberId, 'LOGIN_BANK', 'AU', 'SUCCESS', `balance:${balance}`);

```

Step 10: Return to renderer

```

return { ok: true, balance, balanceFetchedAt: new Date().toISOString() };

```

The result travels back through the IPC bridge, through preload, into the awaiting promise inside `loginBank()`, which then updates the UI.

Step 11: UI updates

`patchBankResult()` (still inside `Dashboard.tsx`) updates the React state, which causes the AU chip to re-render with the new balance string. The toast 'AU refreshed' appears in the bottom-right.

Total elapsed time: 8–15 seconds (depending on bank speed and CAPTCHA difficulty). Files touched: 7 different `.ts` files across `renderer`, `preload`, and `main`. Libraries used: React (for the click), Electron (for IPC), Playwright (for the browser), `better-sqlite3-multiple-ciphers` (for the DB), `keytar` (via field decryption), `googleapis` (for OTP), `Anthropic` (for CAPTCHA).

Part 11 — Glossary

Adapter — A self-contained module that knows how to log into one specific bank or broker. Each adapter implements the `LoginAdapter` interface (`login`, `fetchBalance`, optional `downloadPortfolioReport`, `prepareIpoBid`, `submitPreparedIpoBid`).

AES-256-GCM — A symmetric encryption algorithm. AES = the cipher. 256 = key size in bits. GCM = Galois/Counter Mode, which provides both confidentiality (encryption) and integrity (the auth tag detects tampering). Industry-standard modern choice.

Argon2id — The winner of the 2015 Password Hashing Competition. Slow and memory-hard by design — making brute-force expensive even with GPUs.

asar — Electron's way of packing many small files into one archive for faster loading. The `app.asar` file inside `dist/win-unpacked/resources/` is your compiled code. Files unsafe for asar (native modules, browser binaries) live in `app.asar.unpacked` alongside.

CAPTCHA — Completely Automated Public Turing test to tell Computers and Humans Apart — those squiggly-letter images banks show to slow down automation. AU Bank uses one; we solve it with Anthropic Claude.

Chromium — The open-source web browser engine that Chrome, Edge, Brave, and Electron all use to render web pages.

Electron — A framework that bundles Chromium + Node.js to build desktop apps with web technologies.

Frontend / Backend — Frontend = the visible UI (what you see). Backend = the invisible logic (what does the work).

HTTPS — HTTP over TLS — the encrypted version of the protocol your browser uses. Every bank URL starting with `https://` is encrypted in transit.

IPC (Inter-Process Communication) — Messaging between two separate processes. In Electron, the renderer and main processes use IPC to talk.

Library / Package / Module — Three nearly-synonymous terms for pre-written code you import. In JavaScript-land, all three are typically called 'packages' and live in `node_modules/`.

Main process — The Node.js process that runs the backend logic in Electron. Has full file system / OS access. Lives in `src/main/`.

Migration — A script that updates an existing database to a new schema version (e.g. adds a new column). Designed to be idempotent — running twice has the same effect as running once.

Node.js — JavaScript running outside the browser. Used everywhere on the backend.

npm — Node Package Manager — the standard tool for installing and managing JavaScript libraries. The `package.json` file is npm's manifest.

OAuth — An authentication protocol that lets you grant a third-party app access to your account on another service (e.g. Gmail) without giving away your password. The protocol issues a 'refresh token' that the app stores.

OTP — One-Time Password — a 6-digit code sent to your phone or email, valid for a few minutes. Used as a second factor by most banks.

Playwright — A modern browser-automation library that drives Chrome, Firefox, or Safari from your code.

Preload — A small script that runs in the renderer process before the page loads, but with access to Node APIs. Used to expose a controlled subset of main-process functions to the renderer (via `window.api`).

React — A JavaScript library for building component-based UIs. State changes automatically trigger re-renders.

Renderer process — The Chromium browser tab inside Electron that shows the UI. Sandboxed — cannot directly access the file system. Talks to main via IPC.

Salt — A random value mixed with the password before hashing. Prevents 'rainbow table' attacks. Stored alongside the hashed key — not secret, but essential for reproducing the same key.

SQLCipher — An extension to SQLite that encrypts every page of the database file at rest using AES-256-CBC + HMAC-SHA512. Used here via better-sqlite3-multiple-ciphers.

SQLite — A self-contained file-based database — no server, no network. The whole DB is one .db file.

TOTP — Time-Based One-Time Password — like an OTP but generated locally on your device from a shared secret + the current time. Used by Google Authenticator and broker logins like Zerodha.

TypeScript — JavaScript plus type annotations. Catches type errors at build time. Every .ts file in this project is TypeScript.

Vite — A modern build tool that compiles TypeScript and bundles JavaScript modules. Used by electron-vite for the three-part Electron build.

Vault — Our term for the entire encrypted-data set on disk: vault.db + vault.meta.json + documents/.

Part 12 — How to Add a New Bank Adapter

Suppose you want to add support for IDFC First Bank. Here is the full walk-through. This is the kind of task that, once you have done it once, is repeatable in an hour.

12.1 Pick a code

Every bank has a short uppercase code throughout the codebase: AU, YES, SBI, etc. Pick one for the new bank. For IDFC First, IDFC is the obvious choice.

12.2 Create the adapter file

Copy `src/main/automation/genericBank.ts` to `src/main/automation/idfcBank.ts`. This gives you the boilerplate structure:

```
import { Page } from 'playwright';
import { LoginAdapter, LoginCredentials } from '../browser';

const LOGIN_URL = 'https://my.idfcfirstbank.com/login';

export const idfcBankAdapter: LoginAdapter = {
  code: 'IDFC',
  displayName: 'IDFC First Bank',
  otpMode: 'email', // or 'manual' if user must enter SMS OTP themselves

  async login(page, creds, fetchOtp) {
    await page.goto(LOGIN_URL, { waitUntil: 'domcontentloaded' });
    // ... real login logic goes here
  },

  async fetchBalance(page) {
    // ... scrape balance from the dashboard
    return null;
  },
};
```

12.3 Inspect the real bank's login flow

Open the IDFC site in your normal Chrome. Right-click the user-id field → Inspect. Note the selector — does it have an id? a name? an aria-label? a unique placeholder?

Write down each step:

- Step 1 — User ID field. Selector: `input[name="userid"]`
- Step 2 — Continue button. Selector: `button:has-text("Continue")`
- Step 3 — Password field. Selector: `input[type="password"]`
- Step 4 — Maybe a security image confirmation step.
- Step 5 — Login button. Then OTP screen.
- Step 6 — OTP input field.
- Step 7 — Dashboard URL pattern to wait for.

12.4 Implement login() step by step

```
async login(page, creds, fetchOtp) {
  await page.goto(LOGIN_URL, { waitUntil: 'domcontentloaded' });

  // Step 1: User ID
  await page.fill('input[name="userid"]', creds.username);
  await page.click('button:has-text("Continue")');

  // Step 2: Password
  await page.waitForSelector('input[type="password"]', { timeout: 15_000 });
  await page.fill('input[type="password"]', creds.password);
  await page.click('button:has-text("Login")');

  // Step 3: OTP from Gmail
  await page.waitForSelector('input[name="otp"]', { timeout: 30_000 });
  const otp = await fetchOtp();
  await page.fill('input[name="otp"]', otp);
  await page.click('button:has-text("Submit")');

  // Step 4: Wait for dashboard
  await page.waitForURL(/dashboard/, { timeout: 30_000 });
}
```

12.5 Implement fetchBalance()

On the dashboard, find the savings balance display. Right-click → Inspect. Often it has a unique class like `.balance-value`. Then:

```
async fetchBalance(page) {
  const txt = await page.locator('.balance-value').first().textContent();
  return txt ? txt.trim() : null;
}
```

12.6 Register the adapter

Open `src/main/automation/registry.ts` and add:

```
import { idfcBankAdapter } from './idfcBank';

const BANK_ADAPTERS = {
  AU:    auBankAdapter,
  // ... existing banks ...
  IDFC: idfcBankAdapter,
};
```

12.7 Add the OTP query

In `src/main/email/gmail.ts`, the `OTP_PRESETS` object has a query pattern per bank for finding OTP emails. Add one for IDFC:

```
IDFC_BANK: {
  query: 'from:(@idfcfirstbank.com) subject:(OTP OR login OR verification)',
  otpRegex: /\b(\d{6})\b/,
},
```

And map it in `registry.ts`:

```
const OTP_PRESET_BY_CODE = {
  AU:    OTP_PRESETS.AU_BANK,
  // ...
  IDFC:  OTP_PRESETS.IDFC_BANK,
};
```

12.8 Add to the BANKS constant in the renderer

Open `src/renderer/src/pages/Dashboard.tsx`, find the `BANKS` array, and add 'IDFC':

```
const BANKS = ['AU', 'YES', 'SBI', 'KOTAK', 'ICICI', 'BOB', 'PNB', 'HDFC', 'AXIS', 'IDFC'];
```

Also add an abbreviation in the `BANK_ABBR` map, e.g. `IDFC: 'IF'`, and add a logo PNG to `src/renderer/src/assets/logos/idfc.png` + import in the `BANK_LOGOS` map.

12.9 Build and test

```
npm run build
# or for a packaged installer:
npm run build:win
```

Open the app, add an IDFC bank account to one member, click the IDFC chip. Watch the browser open and execute your selectors. If anything is off, the diagnostic dump in the adapter logs the visible buttons and inputs at the failure point.

12.10 Common gotchas

- **Multi-box OTP fields** — many banks render OTP as 6 separate boxes instead of one input. The Dhan + Angel adapters have helper functions (`fillDigits`) that handle this — copy-adapt from there.
- **iframe wrappers** — some banks load the login form in an `iframe`. `page.frameLocator()` is your tool.
- **Slow load times** — increase `waitForSelector` timeouts to 30s if the bank is slow.
- **Random splash screens** — some banks show a welcome modal that you have to dismiss. Add a `'dismissOverlay'` helper.
- **Re-login required after balance fetch** — some banks log you out after 30 seconds of inactivity. `fetchBalance` has to navigate to the balance page directly, not via menus.

Part 13 — Common Patterns in the Codebase

Once you have read a few files, you will notice the same patterns repeating. Here are the most important ones.

13.1 The IPC handler pattern

Every IPC handler in `src/main/ipc.ts` follows this shape:

```
ipc.handle('something:action', async (_, payload) => {
  try {
    // 1) Validate the payload
    if (!payload?.requiredField) {
      return { ok: false, error: 'missing requiredField' };
    }

    // 2) Do the work
    const result = await doTheWork(payload);

    // 3) Return success with data
    return { ok: true, ...result };
  } catch (e) {
    // 4) Always return a structured error
    return { ok: false, error: e?.message || String(e) };
  }
});
```

The `{ok, error, data}` shape is consistent across every handler. The renderer always knows to check `result.ok` before using `result.data`.

13.2 The encrypted-field pattern

Every sensitive column is stored as a BLOB whose contents come from `encryptField()` and are read back via `decryptField()`:

```
// On write:
db.prepare('UPDATE bank_accounts SET password_enc = ? WHERE id = ?')
  .run(await encryptField(newPassword), bankId);

// On read:
const password = await decryptField(bankRow.password_enc);
```

13.3 The adapter interface pattern

Every bank/broker adapter implements the same TypeScript interface, `LoginAdapter`, defined in `browser.ts`:

```
export interface LoginAdapter {
  code: string;
  displayName: string;
  otpMode?: 'email' | 'manual' | 'totp';

  login(page, creds, fetchOtp): Promise<void>;
  fetchBalance?(page): Promise<string | null>;
  downloadPortfolioReport?(page, creds, fetchOtp): Promise<DownloadedBrokerReport | null>;
  prepareIpoBid?(page, draft): Promise<PreparedIpoBidResult>;
}
```

```
submitPreparedIpoBid?(page, draft): Promise<SubmittedIpoBidResult>;
}
```

This interface is the contract. As long as you implement it correctly, the rest of the system treats your adapter identically to every other adapter. This is the power of abstraction.

13.4 The bus / channel naming convention

IPC channels follow domain:action:

Channel	Domain	Action
vault:unlock	vault	unlock
families:list	families	list
member:fullDetail	member	fullDetail
login:bank	login	bank
backup:status	backup	status
captchaAi:setKey	captchaAi	setKey

13.5 The 'busy state' pattern in the renderer

To prevent double-clicks during async operations, every interactive part uses the busy state pattern:

```
const [busy, setBusy] = useState(null);

async function doSomething() {
  setBusy('something-running');
  try {
    await window.api.something();
  } finally {
    setBusy(null);
  }
}

<button disabled={busy === 'something-running'}>
  {busy === 'something-running' ? 'Working...' : 'Click me'}
</button>
```

13.6 The toast notification pattern

Instead of `alert()` or `console.log`, the renderer uses a small toast queue. `showToast(kind, message)` shows a colored banner in the bottom-right for 5 seconds:

```
showToast('success', 'AU balance refreshed: ■1,23,456');
showToast('error', 'Could not connect to Gmail');
showToast('info', 'Backing up to OneDrive...');
```

13.7 The 'never throw, always return' contract

Inside IPC handlers, we never let an exception bubble up to the renderer. Always catch and return `{ok: false, error: '...'}`. This makes the frontend code simpler — it never needs `try/catch` around an IPC call.

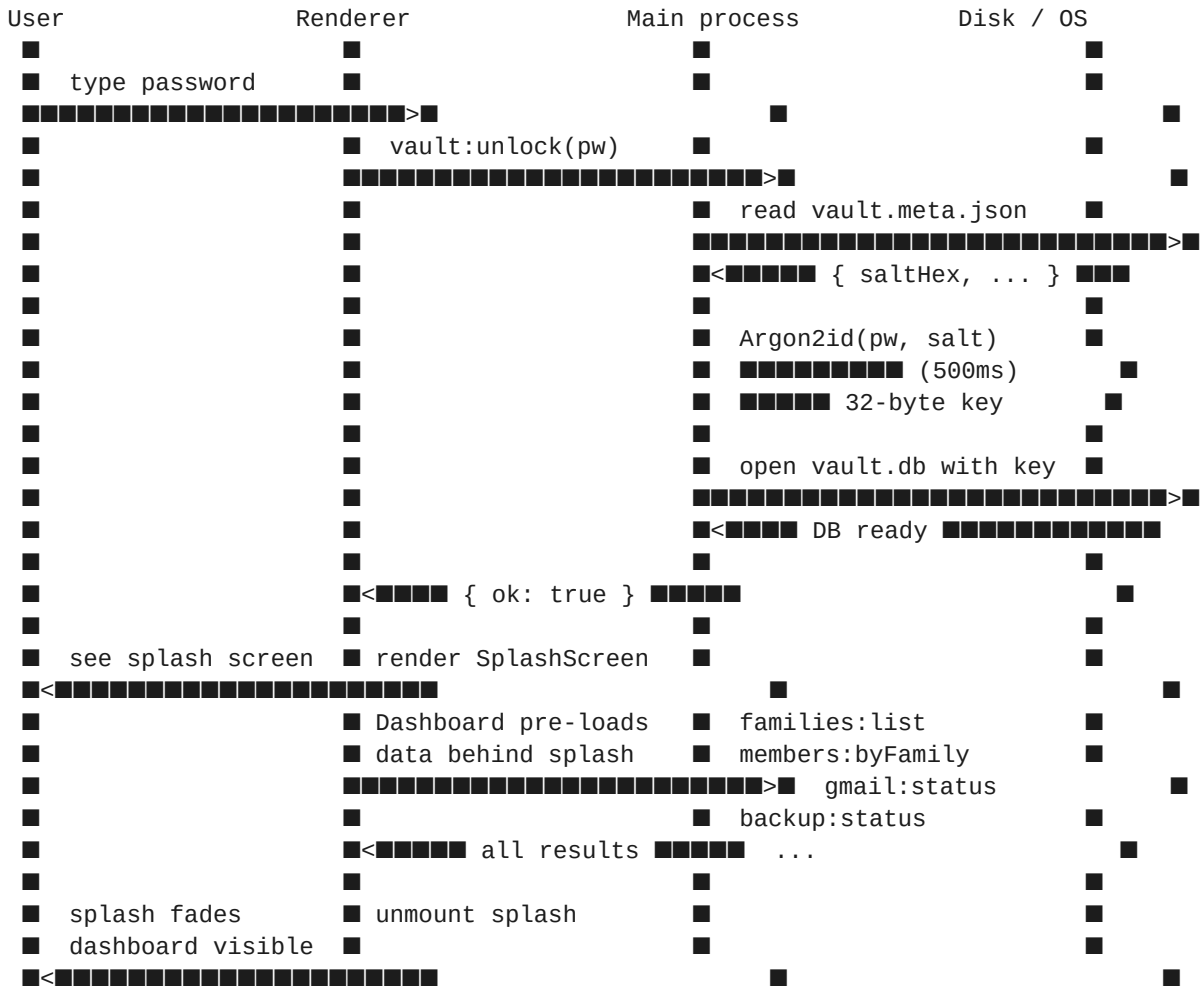
13.8 The 'sensitive data lives in BLOBs, display data lives in TEXT' pattern

Notice the schema: `pan_enc` BLOB (encrypted PAN) but `pan_last4` TEXT (last 4 chars in plaintext for fast list rendering). Same for account numbers and Aadhaar. The plaintext last-4 is safe to leak in screenshots — the full value is always encrypted.

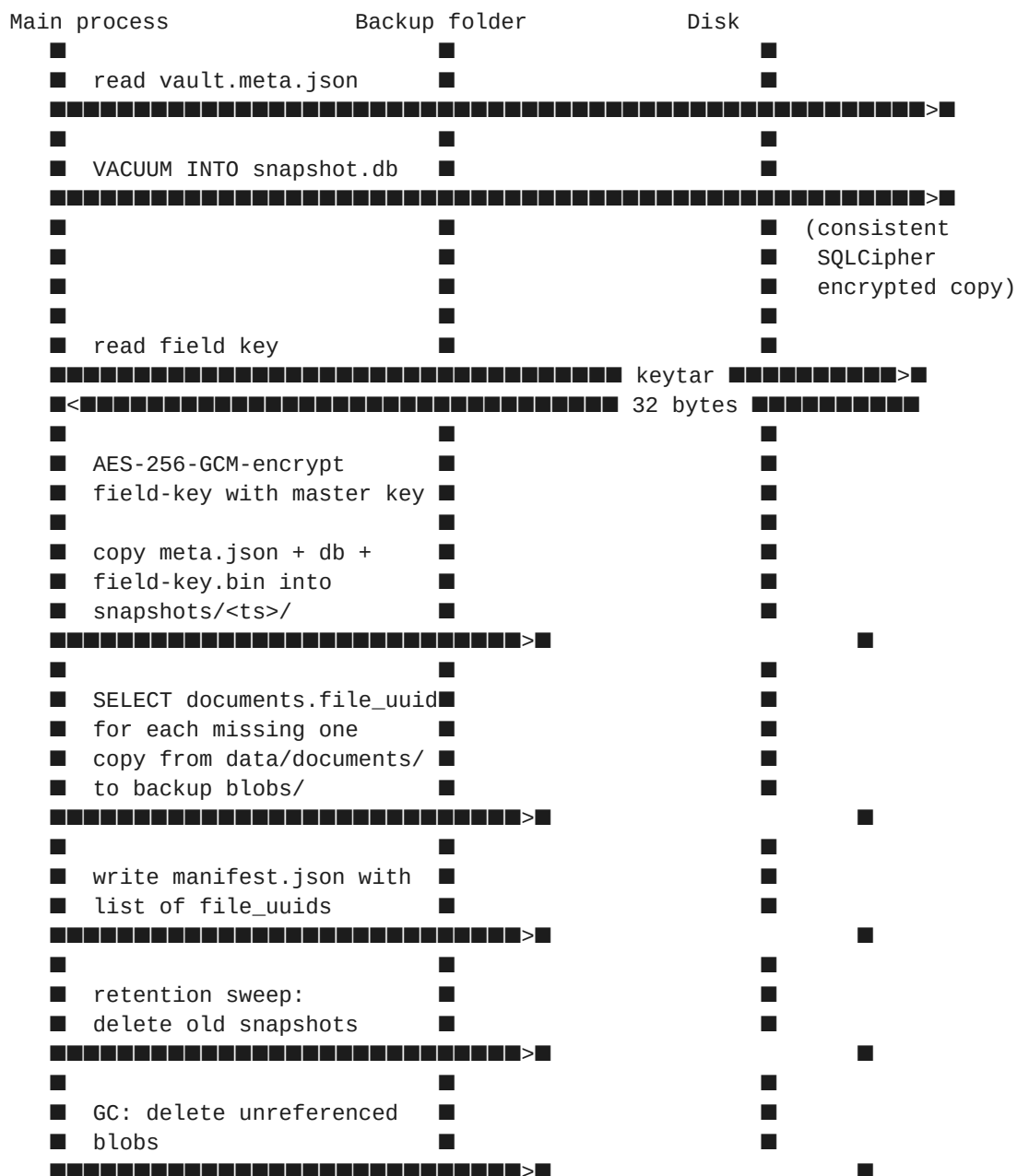
Part 14 — Data Flow Diagrams

Sometimes ASCII diagrams help more than prose. Here are the data flows for the most important operations.

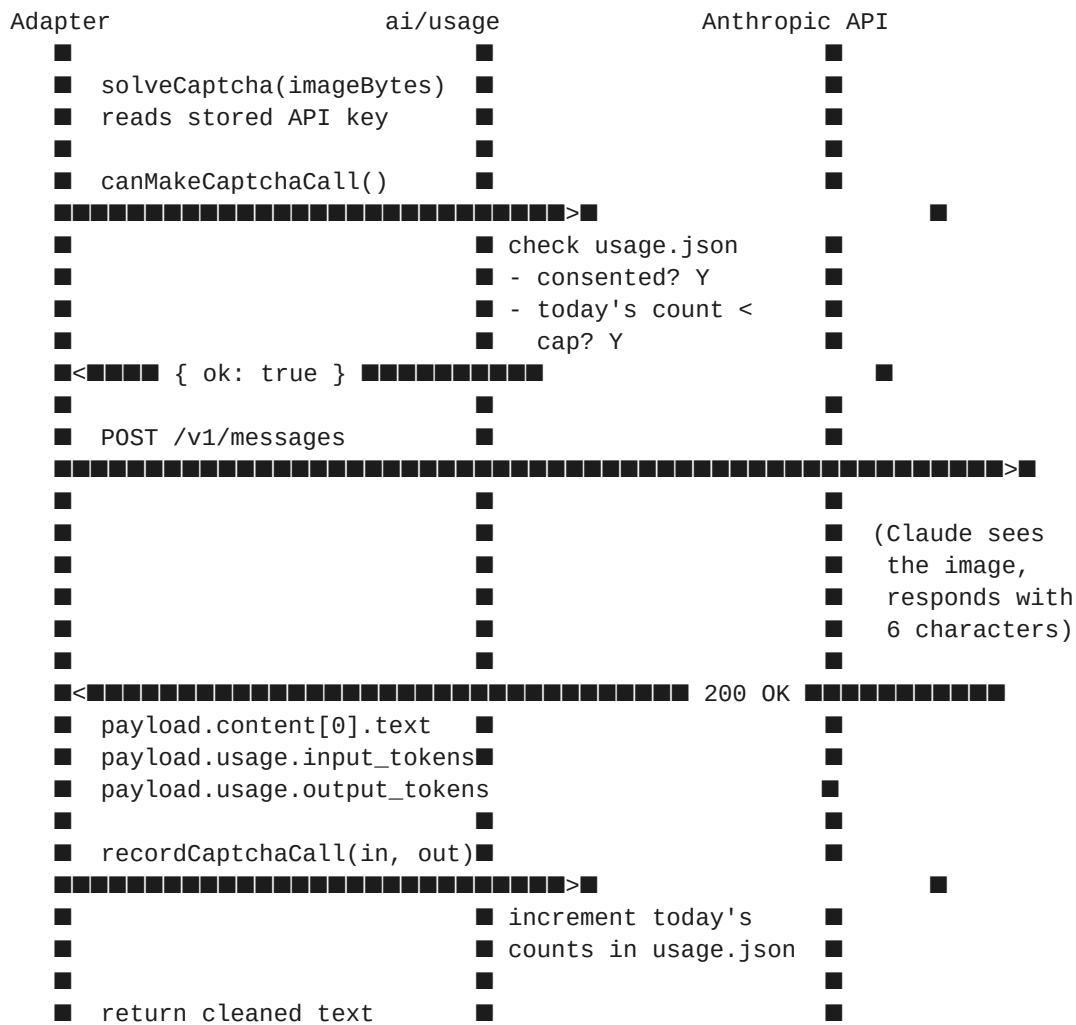
14.1 The unlock flow



14.2 The encrypted backup flow



14.3 The CAPTCHA solve flow with cost gate



Appendix A — Command Cheat Sheet

Building

```
npm install          # download dependencies (~5 min first time)
npm run dev          # run app in development mode (hot-reload)
npm run build        # compile TS to JS in out/
npm run build:win     # build + produce Windows installer in dist/
npm test             # run the test suite
npm run test:watch    # watch mode for test development
```

Backup folder management

```
# Default vault location
%APPDATA%\ipo-manager\data\

# Files in there:
vault.db           # encrypted SQLite database
vault.meta.json    # Argon2 salt + params (not secret)
documents/         # encrypted document blobs
logs/              # automation.log + screenshots
backup.config.json # your chosen backup folder
backup.state.json  # last backup timestamp

# To completely wipe everything (in the app):
#   Backup Settings → Danger zone → "Reset everything..."

# To wipe manually (outside the app):
del /S /Q "%APPDATA%\ipo-manager"
# Then go to Credential Manager and delete the three 'ipo-manager' entries.
```

Inspecting the vault file

```
# The vault file is encrypted – it looks like random bytes without
# the master password.

# To see something:
sqlite3 "%APPDATA%\ipo-manager\data\vault.db" ".tables"
# Result: Error: file is not a database (because SQLCipher rejects it)

# With the right SQLCipher tool and your master password, you can
# inspect it – but normally you should never need to.
```

Diagnosing automation failures

```
# automation.log lives in:
%APPDATA%\ipo-manager\data\logs\automation.log

# Tail the log while running:
type "%APPDATA%\ipo-manager\data\logs\automation.log"

# Look for lines like:
#   [2026-05-20T10:23:45.123Z] [AU_LOGIN] Filled customer id
#   [2026-05-20T10:23:46.456Z] [AU_CAPTCHA] Anthropic raw response: "d3h31d"
#   [2026-05-20T10:23:48.789Z] [AU_LOGIN] OTP received from Gmail
```

Appendix B — Troubleshooting

Symptom: "Wrong password" after typing correctly

Check that Caps Lock is off (passwords are case-sensitive). If you restored a backup recently, make sure you typed the SAME password the backup was created with — not a different one you used on this machine.

Symptom: Clicking a bank does nothing

Check the toast in the bottom-right. Possibilities:

- **"No adapter for X"** → the bank code stored is wrong (e.g. ANGELONE instead of ANGEL). Fix via Edit Member.
- **"Missing credentials"** → user_id or password is empty for this account. Fix via Edit Member.
- **"Could not open browser"** → Chrome/Edge not found on the machine. Install Chrome.
- **Nothing visible** → check automation.log for the actual error.

Symptom: CAPTCHA solving keeps failing

- Check the CAPTCHA AI pill — does it say "ready"?
- Open the modal and check: is consent checked? Is today's count under the cap?
- Check automation.log for "Claude solve skipped: ...".
- Try clicking again — AU's CAPTCHA is randomly hard sometimes; second try often works.

Symptom: Backup pill is yellow or red

- Open Backup Settings.
- Yellow = last backup 24-72h ago. Click "Backup Now".
- Red = >72h ago, or the auto-backup is hitting an error. Check the "Last error" field.

Symptom: "Could not decrypt the field key with the master password" when restoring

This was a bug in the original backup engine. Make sure you are running build version after 2026-05-19 — the latest installer fixes this by including vault.meta.json (the salt) in every snapshot.

Symptom: App is very slow to start

First-launch slowness is normal — Argon2 takes ~500ms by design, and Windows Defender scans the unsigned executable on first run. Subsequent launches should be under 2 seconds plus the splash duration (1.8s).

Where to ask for help

If the toast or log doesn't make the failure obvious, copy the log file (automation.log, last 200 lines) and send it to the developer along with a description of what you were trying to do.

Closing Note

If you have read this far — really read, not just skimmed — you now understand more about how a modern Electron desktop app is structured than most working programmers do. You know:

- What the three layers of a desktop app are and which folder holds each.
- How Argon2id turns a password into a key, and why that matters.
- How Playwright drives a real Chrome window to log into banks.
- Why salt belongs in every backup snapshot.
- What CAPTCHA, OAuth, IPC, AES-GCM, TOTP mean and how they fit together.

The biggest takeaway: software architecture is mostly about *making the right separations*. Renderer vs main. Identity vs credentials. Master key vs field key. Configuration vs data vs code. Each separation is a deliberate choice that protects something or makes something easier to change later.

When you get stuck reading the code, ask the AI:

- "What does this file do?" — start with a one-paragraph summary.
- "What is the data flow from X to Y?" — ask for a step-by-step trace.
- "What are the failure modes of this function?" — ask for what could go wrong.
- "Show me the simplest possible test for this." — get a concrete example.

That is the same way I learned everything in this codebase. The pattern works.

Built with care for you, bhai. May your IPOs be allotted and your balances always above the minimum threshold.